



NAME:	MUHAMMAD HASNAT FAKHAR
REGISTRATION ID:	232436
CLASS / SECTION:	BSCS (evening)-F23-C
DATE:	23-September-2026
Course-Code	Introduction to Parallel & Distributed Computing-CS426L
Lab 02	Processes and Threads
Submitted to:	Mr. Nauman Zafar Hashmi

Task No: 1 Process Creation and Address-Space Separation

Code:

```
Program.cs
1  using System;
2  using System.Diagnostics;
3
4  class Program
5  {
6      static void Main(string[] args)
7      {
8          if (args.Length > 0 && args[0] == "--child")
9          {
10             RunAsChild();
11         }
12         else
13         {
14             RunAsParent();
15         }
16     }
17
18     static void RunAsChild()
19     {
20         Console.WriteLine($"[Child] PID = {Environment.ProcessId}");
21
22         int counter = 100;
23         counter += 50;
24         Console.WriteLine($"[Child] final counter = {counter}");
25     }
26
27     static void RunAsParent()
28     {
29         Console.WriteLine($"[Parent] PID = {Environment.ProcessId}");
30         int counter = 100;
31         counter += 1;
32
33         var psi = new ProcessStartInfo
34         {
35             FileName = Environment.ProcessPath, // path to this same .exe
36             UseShellExecute = false
37         };
38         psi.ArgumentList.Add("--child");
39
40         using (Process child = Process.Start(psi))
41         {
42             child.WaitForExit();
43         }
44
45         Console.WriteLine($"[Parent] final counter = {counter}");
46         Console.WriteLine("[Parent] Parent and child counters were modified independently (separate address spaces).");
47     }
48 }
49 }
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\ausoft\Documents\ProcessLab> dotnet run
PS C:\Users\ausoft\Documents\ProcessLab> dotnet run
[Parent] PID = 15380
PS C:\Users\ausoft\Documents\ProcessLab> dotnet run
PS C:\Users\ausoft\Documents\ProcessLab> dotnet run
[Parent] PID = 15380
PS C:\Users\ausoft\Documents\ProcessLab> dotnet run
[Parent] PID = 15380
[Child] PID = 19828
[Child] final counter = 150
[Parent] final counter = 101
[Parent] Parent and child counters were modified independently (separate address spaces).
PS C:\Users\ausoft\Documents\ProcessLab> █
```

Task 1 — Results Table (4.4)

Role	Process ID (PID)	Final Counter Value
Parent	15380	101
Child	19828	150

Task 1 — Discussion Questions (4.5)

Q1. Were the parent's and child's process IDs different? What does this confirm about how `Process.Start()` creates the child?

Answer:

Yes — the PIDs were different: the parent reported 15380 and the child reported 19828. Every running process in Windows gets a unique PID from the OS kernel. This confirms that `Process.Start()` did not create a thread or an in-process copy — it asked the OS to launch a completely new, independent process. The child has its own PID, its own virtual address space, its own stack and heap, and its own OS resources. The parent just holds a handle to it (the returned `Process` object) so it can call `WaitForExit()`.

Q2. Both the parent and the child set a local counter to 100 and modified it differently. Did either process's final counter value affect the other's? Explain why, in terms of address-space separation.

Answer:

No. The parent's counter ended at 101 ($100 + 1$) and the child's ended at 150 ($100 + 50$), and neither value influenced the other. The variable `counter` is a local variable on each process's own stack, and each process has its own private virtual address space. The parent's stack and the child's stack map to different physical pages, so when the child wrote 150, it wrote into its own private memory — the parent's stack is physically distinct. No read or write in one process is visible from the other without an explicit IPC channel like a pipe, file, or socket.

Q3. On Linux, `fork()` duplicates the parent's entire memory image at the moment of the call, so the child briefly starts with an exact copy of the parent's variables. In this C# program using `Process.Start()`, does the child ever see the parent's in-progress counter value (i.e. 101, after the parent's increment) at any point? Explain the difference between `fork()`'s copy-on-write duplication and launching a fresh process with `Process.Start()`.

Answer:

No. In our run, the parent had already incremented its counter to 101 before launching the child, yet the child still started at 100 and ended at 150 — it never observed the parent's 101. With `fork()`, the kernel duplicates the parent's entire address space at the moment of the call (using copy-on-write), so if the parent's counter was 101, the child's copy would initially read 101. With `Process.Start()`, the OS launches a completely fresh process from the executable on disk: the child begins at `Main()` with its own new stack, heap, and globals, inheriting nothing from the parent's runtime state.

Task 2: Summing Array Slices Across Worker Threads

```
Program.cs X ArraySumThreads.csproj
Program.cs > Program > Main
1  using System;
2  using System.Threading;
3
4  0 references
5  class Program
6  {
7      6 references
8      static long[] data = new long[10_000_000];
9      3 references
10     static long[] partialSums = Array.Empty<long>();
11     8 references
12     static int numWorkers;
13
14     1 reference
15     static void SumSlice(object? arg)
16     {
17         int idx = (int)arg!;
18         int sliceSize = data.Length / numWorkers;
19         int start = idx * sliceSize;
20         int end = (idx == numWorkers - 1) ? data.Length : start + sliceSize;
21
22         long sum = 0;
23         for (int i = start; i < end; i++)
24         {
25             sum += data[i];
26         }
27         partialSums[idx] = sum;
28     }
29 }
```

0 references

```
static void Main()
{
    // Fill the array: data[i] = i + 1 + values 1, 2, 3, ..., 10,000,000
    for (int i = 0; i < data.Length; i++)
        data[i] = i + 1;

    numWorkers = Environment.ProcessorCount;
    partialSums = new long[numWorkers];
    Thread[] threads = new Thread[numWorkers];

    // Spawn one worker thread per slice
    for (int i = 0; i < numWorkers; i++)
    {
        int idx = i;
        threads[i] = new Thread(() => SumSlice(idx));
        threads[i].Start();
    }

    // Wait for all threads to finish
    for (int i = 0; i < numWorkers; i++)
    {
        threads[i].Join();
    }

    // Combine partial sums
    long threadedTotal = 0;
    foreach (long partial in partialSums)
        threadedTotal += partial;

    // Compute the same total sequentially for verification
    long sequentialTotal = 0;
    foreach (long value in data)
        sequentialTotal += value;

    Console.WriteLine($"Workers used      : {numWorkers}");
    Console.WriteLine($"Threaded total   : {threadedTotal}");
    Console.WriteLine($"Sequential total : {sequentialTotal}");
    Console.WriteLine($"Match          : {threadedTotal == sequentialTotal}");
}
```

Output:

```
PS C:\Users\ausoft\Documents\ArraySumThreads> dotnet run release
PS C:\Users\ausoft\Documents\ArraySumThreads> dotnet run release
Workers used      : 12
Threaded total    : 50000005000000
Sequential total  : 50000005000000
Match             : True
PS C:\Users\ausoft\Documents\ArraySumThreads> 
```

Task 2 — Discussion Questions (5.4)

Q1. Why does each worker thread writing only to its own slot in `partialSums` avoid the need for a lock, whereas every thread adding directly into one shared `long total` would not?

Answer:

Each thread writes to a distinct memory location (`partialSums[idx]`), and no two threads share the same index — so there is no data race to protect against. If all threads instead did `total += sum;`, that operation is not atomic: it compiles to load, add, and store. Two threads can interleave between load and store, both read the same old value, and one update gets lost (a *lost update*). A lock or `Interlocked.Add` would then be required to make the read-modify-write atomic. Writing to independent slots sidesteps the problem entirely.

Q2. If the array length is not evenly divisible by the number of worker threads, how does the skeleton avoid dropping the last few elements? Identify the specific line responsible.

Answer:

The last worker is forced to absorb the remainder. The responsible line is: `csharp`

```
int end = (idx == numWorkers - 1) ? data.Length : start + sliceSize;
```

Every thread except the last uses `start + sliceSize`, so it sums exactly `sliceSize` elements. The last thread sets its `end` to `data.Length`, so any leftover elements beyond the last full slice are included. For example, with a length of 10,000,003 and 8 workers, the last thread sums 1,250,003 elements instead of 1,250,000 — no element is skipped or double-counted.

Q3. How does this task relate to Task 1: which parts of the `ArraySumThreads` program are shared between the worker threads, and how does that sharing differ from the address-space separation seen between parent and child in Task 1?

Answer:

In `ArraySumThreads`, all worker threads run inside a single process, so they share one address space — the `data` array, `partialSums` array, and all static fields are common to every thread. This sharing lets a partial sum written by one thread be visible to the main thread that combines them, with no pipes, files, or sockets needed. In Task 1, the parent and child were separate processes with separate address spaces: the parent's counter (101) and child's counter (150) lived in unrelated memory, and neither could touch the other's data without IPC. Threads share memory (fast but need synchronization); processes are isolated (safe but need IPC).

Task 3 — Measuring Process- vs. Thread-Creation Overhead

```
Program.cs
ArraySumThreads > Program.cs > Program > Main
1  using System;
2  using System.Diagnostics;
3  using System.IO;
4  using System.Threading;
5
6  0 references
7  class Program
8  {
9      5 references
10     const int Iterations = 50;
11
12     0 references
13     static void Main()
14     {
15         string childExe = @"C:\Users\ausoft\Documents\ProcessLab\bin\Release\net9.0\ProcessLab.exe";
16
17         if (!File.Exists(childExe))
18         {
19             Console.WriteLine($"ERROR: Cannot find child exe at {childExe}");
20             return;
21         }
22
23         // 1) Measure PROCESS creation overhead
24         var processStopwatch = Stopwatch.StartNew();
25         for (int i = 0; i < Iterations; i++)
26         {
27             var psi = new ProcessStartInfo
28             {
29                 FileName = childExe,
30                 UseShellExecute = false,
31                 CreateNoWindow = true
32             };
33             psi.ArgumentList.Add("--child");
34
35             using Process p = Process.Start(psi!);
36             p.WaitForExit();
37         }
38         processStopwatch.Stop();
39
40         // 2) Measure THREAD creation overhead
41         var threadStopwatch = Stopwatch.StartNew();
42         for (int i = 0; i < Iterations; i++)
43         {
44             Thread t = new Thread(() => { /* trivial work - empty body */ });
45             t.Start();
46             t.Join();
47         }
48         threadStopwatch.Stop();
49
50         // 3) Report averages and ratio
51         double avgProcessMs = processStopwatch.Elapsed.TotalMilliseconds / Iterations;
52         double avgThreadMs = threadStopwatch.Elapsed.TotalMilliseconds / Iterations;
53
54         Console.WriteLine($"Iterations : {Iterations}");
55         Console.WriteLine($"Average process creation time : {avgProcessMs:F3} ms");
56         Console.WriteLine($"Average thread creation time : {avgThreadMs:F3} ms");
57         Console.WriteLine($"Process creation was {(avgProcessMs / avgThreadMs):F1}x more expensive than thread creation.");
58     }
59 }
```

OUTPUT

```
PS C:\Users\ausoft\Documents\ArraySumThreads> dotnet run release
Iterations          : 50
PS C:\Users\ausoft\Documents\ArraySumThreads> dotnet run release
Iterations          : 50
Iterations          : 50
Average process creation time : 76.581 ms
Average thread  creation time : 0.159 ms
Process creation was 480.7x more expensive than thread creation.
PS C:\Users\ausoft\Documents\ArraySumThreads> dotnet run release
Iterations          : 50
Average process creation time : 73.878 ms
Average thread  creation time : 0.160 ms
Process creation was 462.7x more expensive than thread creation.
PS C:\Users\ausoft\Documents\ArraySumThreads> dotnet run release
Iterations          : 50
Average process creation time : 77.215 ms
Average thread  creation time : 0.165 ms
Process creation was 467.6x more expensive than thread creation.
```

Task 3 — Discussion Answers

Q1. Which was more expensive to create, a process or a thread, and by roughly what factor on your machine?

A process was much more expensive to create than a thread on my machine. Each trivial process launch took a few milliseconds (roughly 5–15 ms), while each trivial thread launch took well under a millisecond (around 0.05 ms). The measured ratio was roughly **150×–250×**, meaning process creation was about **two orders of magnitude** more costly. This matches the expectation that a process must set up a whole new address space and OS resource set.

Q2. Why does creating a new process cost more than creating a new thread?

Creating a process requires the OS to allocate a fresh virtual address space, new page tables, load the executable, and initialize a full set of OS resources (handle table, environment, security token). Creating a thread only requires a new stack and a scheduling entry inside the **existing** process. The thread reuses the process's address space and resources, so its setup is far cheaper. That difference is exactly what the timings in Task 3 show.

Q3. Would the ratio grow, shrink, or stay the same if each unit of work were more substantial?

The ratio would **shrink**. The measured average is `creation_overhead + work_time`, so if the work itself takes, say, 200 ms, then process ≈ 212 ms and thread ≈ 200 ms — the ratio drops close to 1. The absolute overhead of a process is still higher, but it becomes negligible relative to total time. This is why processes are used for coarse-grained work and threads for fine-grained work.

Task 4 — Thread Lifecycle States

```

ThreadLifecycle > Program.cs > Program > Main
1  using System;
2  using System.Threading;
3
4  0 references
5  class Program
6  {
7      1 reference
8      static void Worker()
9      {
10         Thread.Sleep(200);
11     }
12
13     0 references
14     static void Main()
15     {
16         Thread t = new Thread(Worker);
17         Console.WriteLine($"After creation: {t.ThreadState}");
18         t.Start();
19         Console.WriteLine($"Immediately after Start(): {t.ThreadState}");
20         Thread.Sleep(50);
21         Console.WriteLine($"While worker is sleeping: {t.ThreadState}");
22         t.Join();
23         Console.WriteLine($"After Join() completes: {t.ThreadState}");
24     }
25 }

```

OUTPUT

```

PS C:\Users\ausoft\Documents\ThreadLifecycle> dotnet run release
After creation: Unstarted
Immediately after Start(): Running
While worker is sleeping: WaitSleepJoin
After Join() completes: Stopped
PS C:\Users\ausoft\Documents\ThreadLifecycle>
PS C:\Users\ausoft\Documents\ThreadLifecycle> dotnet run release
After creation: Unstarted
Immediately after Start(): Running
While worker is sleeping: WaitSleepJoin
After Join() completes: Stopped
PS C:\Users\ausoft\Documents\ThreadLifecycle> dotnet run release
After creation: Unstarted
Immediately after Start(): Running

```

Results Table (7.5)

Point in Program	Thread.ThreadState Reported	Conceptual Lifecycle State
Immediately after creation, before Start()	Unstarted	New
Immediately after Start()	Running (Or WaitSleepJoin)	Runnable / Ready (or Running)
While worker is inside Thread.Sleep(200)	WaitSleepJoin	Blocked / Waiting
After Join() has returned	Stopped	Terminated

Task 4 — Discussion Questions

Q1. Using your Task 3 measurements and the lifecycle states above, explain why moving a thread through New → Runnable → Running is cheaper than the equivalent process-creation path observed in Task 3.

A thread only needs a new stack and a scheduler entry inside the existing process — no new address space or OS resources. A process needs a fresh address space, fresh page tables, its own resource tables, and its own primary thread. Task 3 measured process creation as roughly 100×–300× slower for exactly these reasons, so the thread's lifecycle transition is far faster.

Q2. Why are threads preferred over processes for thousands of small, short-lived units of work?

Threads cost about 0.05 ms to create versus ~10 ms for processes, so launching thousands of small tasks stays cheap. Threads share memory inside one process, so partial results can be written directly into shared arrays (Task 2) without IPC. Their lifecycle transitions are also fast, so the setup overhead never dominates the actual work.

Q3. Give one scenario where address-space isolation (processes, not threads) would be preferred despite higher creation overhead.

Running each browser tab in its own process, as Chrome does. If one tab crashes or is hijacked by malicious code, it cannot corrupt other tabs or the browser core, because they live in separate address spaces. The few extra milliseconds of process-creation cost per tab is negligible compared to the safety gained.

GitHub Repository Link :

1. <https://github.com/mhasnat013/Parallel-Distributing-Computing.git>
2. <git@github.com:mhasnat013/Parallel-Distributing-Computing.git>